

Escalonamento de Processos

Grupo

Felipe Gabriel Kuhn

Eduardo Gonçalves Oliveira

José Vitor Santos da Silva

. CONFERÊNCIA E ANÁLISE DOS RESULTADOS

Foi realizada uma conferência manual dos resultados apresentados pelo simulador. Para cada algoritmo, foi observada a sequência de execução e calculado o tempo de espera de cada processo. Os cálculos foram comparados com os resultados produzidos pelo programa.

Cenário 1 – Conferência manual

Processos:

Processo	CPU	Prioridade
----------	-----	------------

P1	3	1
----	---	---

P2	1	1
----	---	---

P3	2	1
----	---	---

FCFS

Sequência: $P1 \rightarrow P2 \rightarrow P3$.

- P1: espera = 0

- P2: espera = 3

- P3: espera = 4

- Espera média = $(0 + 3 + 4) / 3 = 2,33$

O resultado coincidiu com o programa.

SJF

Sequência: $P2 \rightarrow P3 \rightarrow P1$.

- P2: espera = 0
- P3: espera = 1
- P1: espera = 3
- Espera média = $(0 + 1 + 3) / 3 = 1,33$

O resultado coincidiu com o programa.

Cenário 2 – Efeito do processo longo

Processos:

Processo	CPU	Prioridade
----------	-----	------------

P1	8	1
----	---	---

P2	2	1
----	---	---

P3	1	1
----	---	---

No FCFS, o processo longo P1 é executado primeiro. Com isso, P2 espera 8 unidades de tempo e P3 espera 10 unidades.

No SJF, a sequência é $P3 \rightarrow P2 \rightarrow P1$. Nesse caso, P3 não espera e P2 espera apenas 1 unidade.

Assim, o processo longo fez os processos curtos esperarem mais no FCFS. A espera média foi 6,00 no FCFS contra 1,33 no SJF.

Para o Round Robin, com quantum igual a 1, a CPU é dividida entre os processos. A sequência começa alternando P1, P2 e P3, permitindo que os processos menores sejam concluídos antes de P1 terminar completamente.

A conferência pelo código atual apresentou:

- P1: turnaround = 11; espera = 3
- P2: turnaround = 4; espera = 2
- P3: turnaround = 3; espera = 2
- Espera média = $(3 + 2 + 2) / 3 = 2,33$
- Turnaround médio = $(11 + 4 + 3) / 3 = 6,00$

Portanto, esses são os valores que devem ser considerados no relatório final.

Cenário 3 – Prioridade

Processos:

Processo	CPU	Prioridade
----------	-----	------------

P1	4	3
----	---	---

P2	2	1
----	---	---

P3	3	2
----	---	---

No simulador, quanto menor o valor numérico da prioridade, maior a prioridade.

Assim:

- P2 possui prioridade 1;
- P3 possui prioridade 2;
- P1 possui prioridade 3.

A sequência do algoritmo de Prioridade é:

$P2 \rightarrow P3 \rightarrow P1$

Portanto, P2, que possui a maior prioridade, foi atendido primeiro.

Os tempos de espera são:

- P2: 0
- P3: 2
- P1: 5

Espera média:

$$(0 + 2 + 5) / 3 = 2,33$$

O resultado coincidiu com o programa.

Respostas solicitadas

No cenário 1, qual algoritmo teve menor espera média? Houve empate?

O SJF teve a menor espera média, com 1,33. Não houve empate.

No cenário 2, o processo longo fez os curtos esperarem mais em qual algoritmo?

No FCFS. O processo longo P1 executou primeiro e fez P2 esperar 8 unidades e P3 esperar 10 unidades.

No cenário 3, qual processo tinha a maior prioridade? Ele foi atendido primeiro no algoritmo de prioridade?

O P2 tinha a maior prioridade, pois possuía o menor valor de prioridade, 1. Sim, ele foi atendido primeiro.

O que o Round Robin fez de diferente ao dividir a CPU em várias vezes?

O Round Robin dividiu a CPU em pequenos intervalos de 1 unidade de tempo (quantum). Quando um processo não terminava, ele voltava para o final da fila. Dessa forma, os processos passaram a utilizar a CPU de maneira alternada.

Regra escolhida em cada cenário

Cenário 1: escolheríamos o SJF, pois apresentou a menor espera média, de 1,33.

Cenário 2: escolheríamos o SJF quando o objetivo for reduzir a espera média, pois apresentou 1,33, enquanto o FCFS apresentou 6,00. O Round Robin também apresentou 2,33.

Cenário 3: escolheríamos o SJF ou Prioridade, pois ambos apresentaram espera média de 2,33. A escolha depende do objetivo: SJF favorece processos menores, enquanto Prioridade favorece processos considerados mais importantes.

Relação com o Trabalho 1

A atividade possui relação com o Trabalho 1 porque permite observar, na prática, como o sistema operacional organiza o uso da CPU. Assim como foi observado no Linux,

diferentes regras de escalonamento e prioridades podem alterar a ordem de execução dos processos e o tempo que cada processo permanece aguardando.

Link: <https://github.com/felipegkuhn/Simulador-de-escalonamento>